

Refactoring Targets

- Possible refactoring objectives

1.1 Getting rid of unnecessary imports at the beginning of the file:

We can eliminate the numerous imported packages that are never utilised in some Java files after examining the codes.

```
import java.util.Map;

import warzone.model.*;
import warzone.service.GameEngine;
import warzone.state.Phase;
```

1.2 Simplifying the code in MapService.java

We know that there are some conditions check before the real operation on the object. We can put their condition check into another method and let it return the result of validation to make the current method only process the operation.

```
// condition1: check if more than one country
int l_countryCount = d_gameContext.getCountries().size();
if ( l_countryCount <= 1 ) {
    GenericView.printError("The map should contain more than one country.");
    return false;
}

// condition2: check if each country belongs to one continent
for (Country l_countryTemp : d_gameContext.getCountries().values()) {
    if(l_countryTemp.getContinent() == null) {
        GenericView.printError("Each country should belong to one continent.");
        return false;
    }
}

// condition3: check if more than one continent
Map<Integer, Continent> l_continent = d_gameContext.getContinents();
if ( l_continent.size() <= 1 ) {
    GenericView.printError("The map should contain at least one continent.");
    return false;
}
```

1.3 hard coding in RouterService.java

Through discussion, we find that the Action only has two types, ‘-add’ and ‘-remove’. We can use enumeration to represent it.

```

for(Action l_action: l_actions) {
    //TODO add it in the property file
    String l_actionArray = "-add,-remove";
    if(l_actionArray.indexOf(l_action.getAction()) > -1) {
        String l_command = Arrays.toString(p_commandArray).replace(target: ",", replacement: " ");
        Router l_router = new Router(l_controllerName, l_action.getAction(), l_action.getParameters(), l_command);

        l_routers.add(l_router);
        GenericView.printDebug(p_text: "Add an action to a router");
    }
    else {
        l_routers = new LinkedList<Router>();
        l_routers.add(createErrorRouter(ErrorType.BAD_OPTION.toString()));
        GenericView.printDebug(p_text: "Meet an error when adding an action to a router");
        return l_routers;
    }
}

```

1.4 Simplifying the method in MapService.java:

We can relocate this operation to a different function after looking over the code. A procedure with excessive lines and rationale should be avoided. If so, we can assign the accountability to different strategies.

```

//build the content using StringBuilder
StringBuilder l_map = new StringBuilder();
l_map.append("; map: " + l_fileName);
l_map.append("\n; map made with the 6441 Super Team");
l_map.append("\n; 6441.net 1.0.0.1 ");
l_map.append("\n");

l_map.append("\n[files]");
l_map.append("\npic " + l_fileName + "_pic.jpg");
l_map.append("\nmap " + l_fileName + "_map.gif");
l_map.append("\ncrd " + l_fileName + "europe.cards");
l_map.append("\n");

```

1.5 Change vague method names in MapService.java:

Upon examining the code, we discover that this method's name is not immediately clear. The algorithm is known by its name, the DFS. Its complete name (depth first search) is acceptable.

```

3 usages  ▸ Vrushabh Savani +1
private void innerDFS(int p_cur, int p_low[], int p_disc[],
                    boolean p_stackMember[], Stack<Integer> p_st, LinkedList<Object>[] p_list,
                    List<List<Integer>> p_resList) {

```

1.6 Unclear naming of methods in NeighborService.java

After looking over NeighborService.java, we discover that the functions add() and remove() have ambiguous names in this file. Our recommendation is to modify it to addNeighbor() and removeNeighbor().

```

public boolean add(int p_countryID, int p_neighborCountryID) {
    Map<Integer, Country> l_countries=d_gameContext.getCountries();
    if(!l_countries.containsKey(p_countryID) || !l_countries.containsKey(p_neighborCountryID)) {
        return false;
    }
}

```

```

+ Khushi2111 +1
public boolean remove(int p_countryID, int p_neighborCountryID) {
    Map<Integer, Country> l_countries=d_gameContext.getCountries();
    if(!l_countries.containsKey(p_countryID) || !l_countries.containsKey(p_neighborCountryID)) {
        return false;
    }
}

```

1.7 Never used local members in Country.java

Certain private members, like the map's xPosition and yPosition, are not required. Despite being noted in a few map files, our implementation makes no use of it. As a result, their removal could be considered refactoring.

```

public class Country {
    4 usages
    private int d_countryID;
    4 usages
    private String d_countryName;
    2 usages
    private Player d_owner;
    3 usages
    private int d_xPosition;
    3 usages
    private int d_yPosition;
    2 usages
    private int d_armyNumber = 0;
    4 usages
    private Map<Integer, Country> d_neighbors;
    3 usages
    private Continent d_continent;
}

```

1.8 Never used local members in StartupService.java

Upon reviewing StartupService.java, we see that: a local variable called l_ctr is never utilised in the assigncountries() function. It ought to be taken out.

```
//Looping variables
Country l_country;
Player l_player;
int l_ctr = 0;
int l_playerIndex = 0;

//Loop through each country to assign to a random player
for(Integer l_countryID : l_countryIDs) {
    //Reset the index once each player has been assigned a country
    if(l_playerIndex >= l_playerNames.size()) {
        l_playerIndex = 0;
    }

    l_country = d_gameContext.getCountries().get(l_countryID);
    l_player = d_gameContext.getPlayers().get(l_playerNames.get(l_playerIndex));

    l_country.setOwner(l_player);
    l_player.getConqueredCountries().put(l_country.getCountryID(), l_country);

    //Update the looping variables
    l_playerIndex++;
    l_ctr++;
}
```

1.9 Unsuitable construction of LinkedList in MapService.java

After reading the warning, we discover that the `l_continntAdjList` construct used in `validateSubGraph()` is not a recommended Java language practice.

```
l_continentAdjList = new LinkedList[l_continent.size()];
```

It should be:

```
LinkedList<Object> b = new LinkedList<>();
```

```
LinkedList<LinkedList<Object>> a = new LinkedList<>();
```

```
a.add(b);
```

1.10 Unconventional naming of method in MapService.java

One of the experienced developers in our group has pointed us that the `ifConnected()` method does not adhere to standard naming conventions. The correct one is `isConnected()`.

```
2 usages  harsh-hirapara
public boolean ifConnected(int p_size, LinkedList<Object>[] p_list) {
```

Section 2:

Other 5 refactoring targets

- Actual refactoring targets

2.1 Refactoring architecture using state pattern:

Justification: The game can be split up into several phases, and each phase can be represented by a separate state, allowing a single command to carry out a variety of functions. For instance, we can enter "help" if we need assistance. Because each phase has a unique order that cannot be utilised in other phases, we may override the 'help()' function in each subclass of the state class to provide the player with hints based on the current phase.

Compare: In the first version, we processed player actions using the MVC paradigm. For example, at the start of the game, the player types "help" into the console to request assistance. In the context of the game, there is a field to indicate the current phase. The service's "printHelp()" method will be called by the controller. The issue is that because we have to incorporate all of the various content into the approach, it might become extremely complex and challenging to manage in the future. The code might look like this:

```
public void printHelp(){
    if(d_currentState == "MapEditor"){
        // print all commands that can be used in the MapEditor phase
    }
    else if(d_currentState == "GamePlay"){
        // print all commands that can be used in the GamePlay phase
    }
    // ....
    else {
    }
}
```

When we refactor the architecture using the state pattern, the situation will improve. For each stage of the game, we can make a distinct class and replace the 'printHelp()' functions. After refactoring, the code is simpler to comprehend and maintain because the responsibilities are separated into the appropriate classes. The code might look like this:

```
public abstract class Phase {
    abstract public void printHelp();
}
```

```
public class MapEditor extends Phase {
    public void printHelp(){
        // print all the methods that can be used in the MapEditor phase
    }
}
```

```
public class Reinforcement extends Gameplay {
    public void printHelp(){
        // print all the methods that can be used in the Reinforcement phase
    }
}
```

- Test Cases: there is just 'deploy' order in the first scrum.

✓	DeployOrderTest	5 ms
✓	willNotDeployNegativeArmy	1 ms
✓	willDeploySameArmyAsPool	1 ms
✓	willGenerateDeployOrder	1 ms
✓	willNotDeployToOtherCountry	1 ms
✓	willNotDeployArmyExceedPool	1 ms

2.2 Refactoring architecture using command pattern:

Justification: The game engine uses a round-robin method to carry out player orders that are kept in the player list. The commands are executed at a different time from the location where they are created. Furthermore, we have no desire to discover the precise command that we are currently running. Using a command pattern allows us to carry out the command using a consistent and unambiguous interface.

Compare: In the event that a command pattern is not used, a particular command must be run as soon as it is formed. There will be a terrible code extension. It will be exceedingly challenging to add new instructions, remove existing orders, and alter existing orders. The following code will be used:

```
public void execute(String command){
    switch(command){
        case "deploy":
            CommandService.executeDeploy(command);
        case "advance":
            CommandService.executeAdvance(command);
    }
}
```


When we utilise a command pattern, the circumstances are different. The concrete command can be wrapped into several instances, each of which can be started with an argument. Once they are generated, we can simply return the actual instance of the command, allowing us to run it at a different time and location. Additionally, we are able to do additional operations on it, such as saving or rolling back changes. The code might look like this:

```
public abstract class Order {

    /**
     * current Order Type
     */
    protected OrderType d_orderType;

    /**
     * command which create this order
     */
    protected String d_command;

    /**
     * Current Game Context
     */
    protected GameContext d_gameContext;

    /**
     * get Order Type
     * @return Order Type
     */
    public OrderType getOrderType() {
        return d_orderType;
    }
}
```

```
switch(l_orderName.toLowerCase()){
    case "deploy":
        return createDeployOrder(l_commandInfos);
    case "advance":
        return createAdvanceOrder(l_commandInfos);
    case "bomb":
        return createBombOrder(l_commandInfos);
    case "blockade":
        return createBlockadeOrder(l_commandInfos);
    case "airlift":
        return createAirliftOrder(l_commandInfos);
    case "negotiate":
        return createNegotiateOrder(l_commandInfos);
}
return null;
}
```

Test Cases: there is just 'deploy' order in the first scrum.

✓	✓ DeployOrderTest	5 ms
✓	willNotDeployNegativeArmy	1 ms
✓	willDeploySameArmyAsPool	1 ms
✓	willGenerateDeployOrder	1 ms
✓	willNotDeployToOtherCountry	1 ms
✓	willNotDeployArmyExceedPool	1 ms

2.3 Simplifying Map Editor

Justification: There are distinct operations for adding, deleting, and changing borders, countries, and continents when editing a map. The initial version's code, which used four Boolean fields to represent the stage, is hard to read. Four enumerations will allow us to accomplish that.

Compare: If we don't rearrange our scripts, processing continents, countries, and borders will require a lot of if-else statements. We will also need additional attributes to indicate the currently operated object so that alternative methods may be applied. It will be challenging to extend and maintain the code in the future. Here is the code:

```
//use boolean to record the different parts in file
boolean l_processingFiles = false;
boolean l_processingContinents = false;
boolean l_processingCountries = false;
boolean l_processingBorders = false;

while (l_scanner.hasNextLine()) {
    l_line = l_scanner.nextLine();

    // determine which part it is
    // file part
    if(l_line.equals("[files]")) {

        l_processingFiles = true;
        l_processingContinents = false;
        l_processingCountries = false;
        l_processingBorders = false;

        l_line = l_scanner.nextLine();
    }
    // continents part
    else if(l_line.equals("[continents]")) {

        l_processingFiles = false;
        l_processingContinents = true;
        l_processingCountries = false;
        l_processingBorders = false;

        l_line = l_scanner.nextLine();
    }
}
```


Things will change after refactoring it. We use a single field to indicate the current state. When we want to add something into the map, what we need to do is to add anew enumeration into the class. The code can be following:

```
while (l_scanner.hasNextLine()) {
    l_line = l_scanner.nextLine();

    // determine which part it is
    // file part
    if(l_line.equals("[files]")) {

        l_loadMapPhase = LoadMapPhase.FILES;
        l_line = l_scanner.nextLine();
    }
    // continents part
    else if(l_line.equals("[continents]")) {

        l_loadMapPhase = LoadMapPhase.CONTINENTS;
        l_line = l_scanner.nextLine();
    }
    //countries part
    else if (l_line.equals("[countries]")) {

        l_loadMapPhase = LoadMapPhase.COUNTRIES;
        l_line = l_scanner.nextLine();
    }
}
```

Test Cases:

✓	MapServiceTest	17 ms
✓	validateMap1	5 ms
✓	validateMap2	0 ms
✓	validateMap3	3 ms
✓	validateMap4	1 ms
✓	validateContinentMap1	1 ms
✓	validateContinentMap2	0 ms
✓	testEditMap	7 ms

2.4 Erasing the hard coding

Justification: Every programme should avoid using hard coding since it would produce confusion when we constantly change our codes. Identifying the portion that changes regularly or will change in the future is a useful solution. This can be added to the configuration files. If this section has to be changed, it can be done without modifying code and directly in config

file.

Compare: We divide commands into two categories: basic commands and sophisticated commands, as we discuss in section 2.4. Our initial code is written by hand using hard coding. In order to determine the type of command, we create two strings with all of the commands in them. We next verify if the command entered is present in the strings. Here is the code:

```
String l_complexCommand = "editcontinent,editcountry,editneighbor,gameplayer";  
String l_simpleCommand = "loadmap,editmap,savemap,assigncountries,validatemap,showmap,help,play,reboot,startup,mapeditor";
```

To avoid hard coding, we can put them into the configuration file. It is easier to modify the code in the future, and we do not need to change our codes when we add new commands.

```
#Command  
simpleCommand=,loadmap,editmap,savemap,assigncountries,validatemap,showmap,help,qamode,play,next,reboot,startup,mapeditor,  
complexCommand=,editcontinent,editcountry,editneighbor,gameplayer,
```

✓ RouterServiceTest	15 ms
✓ testValidatemap	1 ms
✓ testLoadMap1	3 ms
✓ testLoadMap2	1 ms
✓ testLoadMap3	0 ms
✓ testAssigncountries	1 ms
✓ testMissingCommand1	1 ms
✓ testMissingCommand2	0 ms
✓ testParseCommand1	2 ms
✓ testParseCommand2	1 ms
✓ testParseCommand3	1 ms
✓ testParseCommand4	0 ms
✓ testParseCommand5	1 ms
✓ testParseCommand6	0 ms
✓ testParseCommand7	1 ms
✓ testNoSuchCommand1	0 ms
✓ testNoSuchCommand2	1 ms
✓ testSaveMap	1 ms
✓ testEditMap	0 ms

2.5 Aggregate separate actions into one method

Justification: We will need to update both the owner and the conquered list in Player whenever we switch out a country's player. We choose to rework this because, should someone neglect to complete one of the steps, it could lead to data discrepancies.

Compare: Prior to restructuring, we had to add the nation to the list of conquered Countries and call the `setOwner()` function each time we set a player.

```
l_country.setOwner(l_player);  
l_player.getConqueredCountries().put(l_country.getCountryID(), l_country);
```

```
public void setOwner(Player p_owner) {  
    this.d_owner = p_owner;  
}
```

After refactoring, we will do these two tasks into `setOwner`. Also, we have more validation checked before actions.

```
LoveBorad15 +2  
public void setOwner(Player p_owner) {  
    if (d_owner != null)  
        d_owner.getConqueredCountries().remove(this.getCountryID(), this);  
    this.d_owner = p_owner;  
    if(p_owner != null)  
        p_owner.getConqueredCountries().put(this.getCountryID(), this);  
}
```

Test Cases:

```
✓ CountryTest 1 ms  
  ✓ testSetOwnerSuccess 1 ms
```